

Guía de instalación y despliegue

Documento oficial de instalación y puesta en marcha del **Taller CRUD Tienda (POO)**. Siga estos pasos en orden para dejar el sistema funcionando en su computador.

1. Requisitos previos

Componente	Versión usada en el desarrollo	Qué se necesita
Sistema operativo	Windows	Cualquier sistema con Python y Node.js
Python	3.12.5	Python 3.10 o superior
Node.js	v23.2.0	Node.js 20 o superior (viene con npm)
Navegador	Chrome / Edge / Firefox	Cualquiera actual

No hace falta instalar una base de datos: SQLite viene incluida en Python, y el archivo de la tienda (database/tienda.db) **se crea solo la primera vez que se ejecuta el backend**, con datos de demostración.

2. Estructura del proyecto

P00/	
├─ backend/	
│ └─ app.py	# Aplicacion Flask y punto de arranque
│ └─ models/	# Modulo 1: clases P00 (Modelo)
│ └─ controllers/	# Modulo 2: logica CRUD (Controlador)
│ └─ data/	# Modulo 3: persistencia SQLite
│ └─ routes/	# Modulo 4: API REST
├─ frontend/	# Modulo 5: Vista (Vite + React)
├─ database/	
│ └─ schema.sql	# Esquema de las tablas
│ └─ seed.sql	# Datos iniciales de demostracion
├─ docs/	
│ └─ diagramas/	# Diagramas de disenio
└─ instalacion_despliegue.md	# Este documento

3. Instalación

3.1. Backend (Python + Flask)

1. Abra una terminal en la carpeta raíz del proyecto.
2. Instale Flask (única dependencia externa del backend):

```
pip install flask
```

3. (Opcional) Verifique que todo el backend está correcto:

```
python -m backend.demo_persistencia
```

Debe mostrarse una secuencia de resultados y, al final, la reapertura de los datos. Estos scripts solo dejan el archivo `database/tienda.db` con datos de demostración.

3.2. Frontend (Vite + React)

1. Abra una segunda terminal en la carpeta `frontend`.
2. Instale las dependencias del frontend (la primera vez tarda unos minutos):

```
npm install
```

4. Despliegue (puesta en marcha)

Encienda las dos mitades del sistema. Cada una en su propia terminal:

Terminal 1 — el backend

Desde la carpeta raíz del proyecto:

```
python -m backend.app
```

- Respuesta esperada: el mensaje de Flask indicando que corre en `http://127.0.0.1:5000`.
- La primera vez, si el archivo de base de datos no existe, se crea automáticamente con las tablas (`schema.sql`) y los datos de demostración (`seed.sql`).
- No cierre esta terminal mientras use el sistema.

Terminal 2 — la pantalla (Vista)

Desde la carpeta `frontend`:

```
npm run dev
```

- Respuesta esperada: Vite indica que el servidor está listo en `http://localhost:5173`.
- Abra esa dirección en el navegador.

El puerto 5173 reenvía automáticamente las direcciones `/api/*` al backend del puerto 5000 (proxy). El navegador solo "habla" con la Vista.

5. Uso del sistema

Desde la barra superior de la pantalla se elige la sección:

Pestaña	Funcionalidad
Categorías	Crear, listar, editar y eliminar categorías
Productos	Crear, listar, editar y eliminar productos; filtrar por categoría
Clientes	Crear, listar, editar y eliminar clientes (con puntos y "frecuente")
Pedidos	Crear pedido (cliente + fecha), agregar productos, confirmar y cambiar estado

Los datos quedan guardados en `database/tienda.db` y se conservan al cerrar los servidores.

6. Prueba de humo (verificación rápida)

Con los dos servidores encendidos, verifique que la página responde y que la API devuelve datos:

1. En el navegador abra `http://localhost:5173` y confirme que aparece el título y las cuatro pestañas (Categorías, Productos, Clientes, Pedidos) con los datos de ejemplo.
2. Desde una terminal, consulte el estado del backend:

```
curl http://localhost:5000/api/estado
```

Respuesta esperada: `{"servicio": "Tienda CRUD", "estado": "operativo"}`.

3. Confirme que las peticiones también funcionan a través de la pantalla (proxy):

```
curl http://localhost:5173/api/categorias
```

Respuesta esperada: la lista de categorías en formato JSON.

7. Solución de problemas

Problema	Causa probable	Solución
"Address already in use" al iniciar Flask	Puerto 5000 ocupado	Cierre el proceso que lo usa o cambie el puerto en <code>backend/app.py</code>
"Port 5173 is already in use" en Vite	Puerto 5173 ocupado	Vite ofrece abrir otro puerto automáticamente; acepte el propuesto
El navegador muestra errores de red al cargar	El backend está apagado	Encienda la Terminal 1 antes de recargar la página
Datos "sucios" que se quieren reiniciar	Base de datos con datos de prueba	Detenga el backend, borre <code>database/tienda.db</code> y vuelva a iniciarlo (se regenera desde <code>seed.sql</code>)

Problema	Causa probable	Solución
Las tildes se ven raras al imprimir en consola	Codificación de la terminal	Los archivos usan UTF-8; configure la terminal en UTF-8

8. Notas del alcance académico

- El sistema no incluye autenticación de usuarios ni control de sesiones (fuera del alcance del taller).
- Los precios se manejan en pesos sin decimales; las operaciones internas usan `float`.
- La base de datos es un archivo local (SQLite) y no requiere servidor de base de datos.
- Ante errores de validación, las respuestas de la API devuelven un mensaje descriptivo y un código HTTP estándar (400/404), y la interfaz muestra ese mensaje.